ODOO DEVELOPER PROGRAM

ORM Methods

Odoo 17 Development Tutorials

Ahmed Muhumed

July 25, 2026

Outline

Introduction to ORM Methods

What is ORM?

- The ORM stands for **Object Relational Mapping**.
- It is a programming technique that allows developers to interact with a database using **Python objects and methods** instead of writing **SQL queries directly**.
- In Odoo, the ORM acts as a **bridge** between the Python application and the PostgreSQL database.
- Instead of writing:

```sql
INSERT INTO student_student (name, age) VALUES ('Ahmed', 22);
```

- we simply write:

```python
self.env['student.student'].create({
    'name': 'Ahmed',
    'age': 22,
})
```

What Does "Object Relational Mapping" Mean?

- The term consists of three parts.
  1. **Object** - an object is an instance of a Python class.

```
student = self.env['student.student'].browse(1)
```

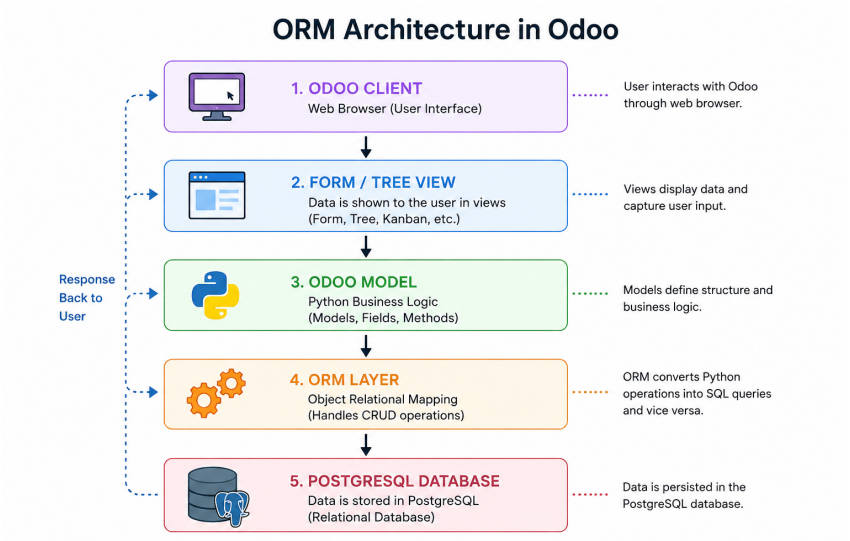
  - Here, ***student*** is a Python object representing a student.
  2. **Relational** - a relational database stores data in tables that are connected using relationships.

Students	Classes
Mohamed	Grade 12

  3. **Mapping** - mapping means connecting two different worlds.

Python Object ⇒ Odoo ORM ⇒ PostgreSQL Table

ORM Architecture



Why is ORM Important?

- Without an ORM, developers would need to:
  - Write SQL for every operation.
  - Manage database connections manually.
  - Convert database rows into Python objects.
  - Prevent SQL injection.
- Although powerful, the ORM has limitations.
  - Less control over complex SQL queries.
  - Very large reports may require raw SQL.
  - Advanced database tuning sometimes needs direct SQL.

Common ORM Methods

- create
- write
- unlink
- copy
- exists
- search
- read
- search_read
- search_count
- name_create
- default_get
- name_search
- get_view()
- ensure_one()
- filtered()
- mapped()
- sorted()
- grouped()
- fields_get()
- get_metadata

`Create()` ORM Method

Create() Method

- The create() method is an ORM (Object Relational Mapping) method used to create one or more new records in an Odoo model.
- The create() method returns a **recordset** containing the newly created record.

Method Syntax

- The standard syntax of the create() method is:

```
@api.model
def create(self, vals):
    return super().create(vals)
```

- This method has four important parts:
  1. **@api.model**
  2. **create**
  3. **self**
  4. **vals**

Breaking Down the Method Signature

- **`@api.model`**: Indicates that the method is executed at the model level. The @api.model decorator tells Odoo that the method belongs to the model, not to an existing record.
- **`create`**: The ORM method responsible for creating records.
- **`self`**: The model (`student.student`), not a specific student record. If we are using `student.student` model, self does not represent a student record. Instead, `self`, represents the model `student.student`.

```
print(self)
```

Output is:

```
student.student()
```

Breaking Down the Method Signature

- **`vals`**: A dictionary containing the values to be stored in the database.

```
name = fields.Char()
birth_date = fields.Date()
gender = fields.Selection(...)
```

```
vals = {
    'name': 'Ahmed Mohamed',
    'birth_date': date(2005, 5, 10),
    'gender': 'male',
}
```

  - Each **key** corresponds to a field name in your model, and
  - Each **value** is the data to store.
  - The ORM maps each key to its corresponding field.

What Does `create()` Return?

- The **`create()`** method returns a recordset.

```
student = self.env['student.student'].create({
    'name': 'Ahmed',
})
```

- Now, if we try to print the **`student`** using **`print(student)`**, we are getting this

```
student.student(15,)
```

@API.MODEL VS @API.MODEL_CREATE_MULTI

There are two common decorators for create().

1. **@api.model**: Used for creating a single record.

```python
@api.model
def create(self, vals):
```

   - **vals** is a dictionary.

```python
student = self.env['student.student'].create({
    'name': 'Ahmed Mohamed',
})
```

2. **@api.model_create_multi**: Used for creating multiple records efficiently.

```python
@api.model_create_multi
def create(self, vals_list):
```

@api.model VS @api.model_create_multi

- **vals_list** is a list of dictionaries, i.e.,

```
[
  {'name': 'Ahmed'},
  {'name': 'Amina'},
]
```

Method Call

- A method call is when we **execute** the method.

```
student = self.env['student.student'].create({
    'name': 'Ahmed Mohamed',
    'email': 'ahmed@example.com',
})
```

- Here, **.create({...})** is the **method call**.

`write()` ORM Method

`write()` Method

- The **`write()`** method is an ORM method used to **update** one or more existing records in an Odoo model.
- Unlike `create()`, `write()` works on an existing recordset.
- The `write()` method returns a boolean value: `True` when the write succeeds.

Method Syntax

- The standard syntax of the write() method is:

```
def write(self, vals):
    return super().write(vals)
```

- This method has three important parts:
  1. **write**
  2. **self**
  3. **vals**

Breaking Down the Method Signature

- **`write`**: The ORM method responsible for updating records.
- **`self`**: A recordset of one or more records that will be updated. If we browse student ID 15, `self` represents that student record.

```
student = self.env['student.student'].browse(15)
print(student)
```

Output is:

```
student.student(15,)
```

- If `self` contains many records, `write()` updates **all** of them with the same values.

Breaking Down the Method Signature

- **`vals`**: A dictionary containing the field values to update.

```
vals = {
    'name': 'Ahmed Muhumed',
    'email': 'ahmed@example.com',
    'age': 23,
}
```

  - Each **key** is a field name on the model.
  - Each **value** is the new data to store.
  - Fields that are not listed in `vals` stay unchanged.
  - The ORM maps each key to its corresponding database column.

What Does `write()` Return?

- The **`write()`** method returns `True` on success.

```
student = self.env['student.student'].browse(15)
result = student.write({
    'name': 'Ahmed Muhumed',
})
print(result)
```

Output is:

```
True
```

- The record itself is updated in the database.
- You can continue using the same recordset after writing.

Updating One Record vs Many Records

1. **One record:**

```
student = self.env['student.student'].browse(15)
student.write({'age': 23})
```

2. **Many records (same values for all):**

```
students = self.env['student.student'].browse([15, 16, 17])
students.write({'active': True})
```

- One `write()` call on a multi-recordset is more efficient than looping and writing each record separately.
- Inverse and computed fields may also be recomputed after `write()`.

Method Call and Override Pattern

Method Call

- A method call is when we **execute** the method.

```
student.write({
    'name': 'Ahmed Muhumed',
    'email': 'ahmed@example.com',
})
```

- Here, `.write({...})` is the **method call**.

Common Override Pattern

```
def write(self, vals):
  # custom logic before saving
  if 'email' in vals:
    vals['email'] = vals['email'].lower()
  return super().write(vals)
```

`unlink()` ORM Method

`unlink()` Method

- The **`unlink()`** method is an ORM method used to **delete** one or more existing records from an Odoo model.
- In database terms, `unlink()` is similar to SQL `DELETE`.
- The method returns a boolean value: `True` when deletion succeeds.

Method Syntax

- The standard syntax of the unlink() method is:

```
def unlink(self):
    return super().unlink()
```

- This method has two important parts:
  1. **`unlink`**
  2. **`self`**

Breaking Down the Method Signature

- **`unlink`**: The ORM method responsible for deleting records.
- **`self`**: A recordset of one or more records that will be deleted.

```
student = self.env['student.student'].browse(15)
print(student)
```

Output is:

```
student.student(15,)
```

- `unlink()` does **not** take a `vals` dictionary.
- It only needs the recordset that should be removed.

What Does `unlink()` Return?

- The **`unlink()`** method returns `True` on success.

```
student = self.env['student.student'].browse(15)
result = student.unlink()
print(result)
```

Output is:

```
True
```

- After `unlink()`, the record no longer exists in the database.
- Related one2many / many2many cleanup is handled by the ORM according to the field `ondelete` behavior.

Deleting One Record vs Many Records

1. **One record:**

```
student = self.env['student.student'].browse(15)
student.unlink()
```

2. **Many records:**

```
students = self.env['student.student'].search([
    ('active', '=', False),
])
students.unlink()
```

- Prefer one `unlink()` on a recordset over deleting records in a Python loop.
- Access rights and record rules still apply before deletion.

Method Call and Override Pattern

Method Call

```
student.unlink()
```

- Here, `.unlink()` is the **method call**.

Common Override Pattern

```
def unlink(self):
  for record in self:
    if record.state == 'done':
      raise UserError("Done students cannot be deleted.")
  return super().unlink()
```

Important Notes About `unlink()`

- Deletion is permanent for normal models (unless you use archive/`active`).
- Many business models prefer **archiving** instead of deleting:

```
1 student.write({'active': False})  # soft delete / archive
```

- Foreign-key constraints may raise errors if other records still depend on the record you are deleting.
- Always check business rules before calling `unlink()` in production code.

`copy()` ORM Method

`copy()` Method

- The **`copy()`** method is an ORM method used to **duplicate** an existing record.
- It creates a new record by copying field values from the current record.
- The method returns a recordset containing the newly created (duplicated) record.

Method Syntax

- The standard syntax of the copy() method is:

```
def copy(self, default=None):
    return super().copy(default)
```

- This method has three important parts:
  1. **`copy`**
  2. **`self`**
  3. **`default`**

Breaking Down the Method Signature

- **`copy`**: The ORM method responsible for duplicating records.
- **`self`**: The source recordset to copy from. In normal usage, `self` is usually **one** record.
- **`default`**: An optional dictionary of values that override the copied values.

```
default = {
    'name': 'Ahmed Mohamed (Copy)',
}
```

- Keys in `default` replace the values that would otherwise be copied.
- Fields not listed in `default` are copied from the original record (unless marked `copy=False`).

What Does `copy()` Return?

- The **`copy()`** method returns a new recordset.

```
student = self.env['student.student'].browse(15)
new_student = student.copy()
print(new_student)
```

Output is (example):

```
student.student(28,)
```

- A new database row is created.
- The original record remains unchanged.

Using the default Argument

- You can change some values while copying.

```
student = self.env['student.student'].browse(15)
new_student = student.copy({
    'name': 'Amina Mohamed',
    'email': 'amina@example.com',
})
```

- Everything else is copied from student 15.
- This is useful for “Duplicate” buttons in the UI.

Which Fields Are Copied?

- By default, most stored fields are copied.
- A field can opt out of copying:

```
email = fields.Char(copy=False)
registration_code = fields.Char(copy=False)
```

- Fields with `copy=False` become empty / default on the new record.
- One2many lines may also be copied depending on the field definition.
- Unique constraints may force you to override values through `default`.

Method Call and Override Pattern

Method Call

```
new_student = student.copy({'name': 'Ahmed (Copy)'})
```

Common Override Pattern

```
def copy(self, default=None):
    default = dict(default or {})
    default.setdefault('name', f"{self.name} (Copy)")
    return super().copy(default)
```

`exists()` ORM Method

`exists()` Method

- The **`exists()`** method checks whether the records in a recordset still exist in the database.
- It is useful when a recordset may contain deleted or invalid IDs.
- The method returns a recordset containing only the records that still exist.

Method Syntax

- The standard syntax of the exists() method is:

```
1 def exists(self):
2   return super().exists()
```

- This method has two important parts:
  1. **`exists`**
  2. **`self`**

Breaking Down the Method Signature

- **exists**: The ORM method that filters a recordset to real DB rows.
- **self**: The recordset you want to validate.

```
students = self.env['student.student'].browse([15, 16, 9999])
print(students)
```

Output is:

```
student.student(15, 16, 9999)
```

- `browse()` does not check the database immediately.
- ID `9999` may not exist, but it is still inside the recordset.

What Does `exists()` Return?

- `exists()` returns a recordset of only the records that exist.

```
students = self.env['student.student'].browse([15, 16, 9999])
real_students = students.exists()
print(real_students)
```

Output is (example):

```
student.student(15, 16)
```

- ID `9999` was removed because it is not in the database.

Checking a Single Record

- You can also test one record in an `if` statement.

```
student = self.env['student.student'].browse(15)
if student.exists():
    print("Student is still in the database")
else:
    print("Student was deleted")
```

- An empty recordset is falsy in boolean context.
- This pattern avoids crashing when a record was already unlinked.

When Should You Use `exists()`?

- After long-running operations where records may have been deleted.
- When you received IDs from the UI, RPC, or another model.
- Before writing or unlinking a recordset that might be stale.

```
records = self.env['student.student'].browse(ids).exists()
records.write({'active': True})
```

- `exists()` is a safety method: it does not create or update data.

Method Call

Method Call

```
1 real_students = students.exists()
```

- Here, `.exists()` is the **method call**.
- No dictionary argument is required.
- The return value is always a recordset (possibly empty).

`search()` ORM Method

`search()` Method

- The **`search()`** method is an ORM method used to **find records** that match a domain (filter condition).
- It is one of the most used ORM methods in Odoo.
- The method returns a recordset of matching records.

Method Syntax

- The standard syntax of the search() method is:

```
@api.model
def search(self, domain, offset=0, limit=None, order=None):
    return super().search(domain, offset=offset, limit=limit, order=order)
```

- Important parts: **`domain`**, **`offset`**, **`limit`**, **`order`**.

Breaking Down the Method Signature

- **`@api.model`**: Search runs at model level (not on one existing record).
- **`self`**: Represents the model, e.g. `student.student()`.
- **`domain`**: A list of conditions used to filter records.

```
domain = [('age', '>=', 18), ('gender', '=', 'male')]
```

- Each condition is a tuple: `(field_name, operator, value)`.
- Multiple conditions are combined with AND by default.

Breaking Down Domain Operators

Common operators used inside a domain:

- = equal
- != not equal
- >, >=, <, <= comparisons
- `like` / `ilike` text search (`ilike` is case-insensitive)
- `in` / `not in` membership
- `child_of` hierarchical search

```
[('name', 'ilike', 'ahmed')]
[('id', 'in', [15, 16, 17])]
['|', ('gender', '=', 'male'), ('age', '<', 18)]
```

- `'|'` means OR, `'&'` means AND, `'!'` means NOT.

Offset, Limit, and Order

- **offset**: Skip the first N records (used for pagination).
- **limit**: Return at most N records.
- **order**: SQL-like ordering string.

```
students = self.env['student.student'].search(
    [('active', '=', True)],
    offset=10,
    limit=5,
    order='name asc, id desc',
)
```

- This returns active students 11–15 when sorted by name.

What Does `search()` Return?

- The **`search()`** method returns a recordset.

```
students = self.env['student.student'].search([
    ('name', 'ilike', 'ahmed'),
])
print(students)
```

Output is (example):

```
student.student(15, 22, 31)
```

- If nothing matches, it returns an empty recordset: `student.student()`.
- An empty recordset is falsy in an `if` condition.

Method Call and Practical Example

Method Call

```
students = self.env['student.student'].search([
    ('age', '>=', 18),
    ('active', '=', True),
], limit=20, order='age desc')
```

- Here, `.search(...)` is the **method call**.
- Always prefer domains over building raw SQL for normal queries.
- Record rules and access rights are applied automatically.

`read()` ORM Method

`read()` Method

- The **`read()`** method is an ORM method used to **fetch field values** from a recordset as plain Python dictionaries.
- It is commonly used by the web client and RPC calls.
- The method returns a **list of dictionaries**, not a recordset.

Method Syntax

- The standard syntax of the read() method is:

```
def read(self, fields=None, load='_classic_read'):
    return super().read(fields=fields, load=load)
```

- Important parts: **`self`**, **`fields`**, **`load`**.

Breaking Down the Method Signature

- **`self`**: The recordset whose values you want to read.
- **`fields`**: An optional list of field names to fetch.

```
fields = ['name', 'age', 'email']
```

- If `fields` is omitted, Odoo reads all readable fields.
- Reading only needed fields is faster and cleaner.
- **`load`**: Controls how relational values are returned (classic read vs plain IDs).

What Does `read()` Return?

- `read()` returns a list of dictionaries.

```
student = self.env['student.student'].browse(15)
data = student.read(['name', 'age'])
print(data)
```

Output is (example):

```
[{'id': 15, 'name': 'Ahmed', 'age': 22}]
```

- Each dictionary represents one record.
- The key `id` is always included.

Reading Many Records

```
students = self.env['student.student'].browse([15, 16])
data = students.read(['name', 'email'])
```

Output is (example):

```
[
  {'id': 15, 'name': 'Ahmed', 'email': 'ahmed@example.com'},
  {'id': 16, 'name': 'Amina', 'email': 'amina@example.com'},
]
```

- Useful when you need serializable data (JSON / RPC / API).
- For normal business logic inside Odoo, prefer recordset access: `student.name`.

READ() VS RECORDSET ACCESS

1. **Recordset access** (preferred in Python business code):

```
name = student.name
age = student.age
```

2. **read()** (preferred for RPC / exporting dict data):

```
data = student.read(['name', 'age'])[0]
```

- Recordset access keeps ORM features (prefetch, cache, relational browsing).
- `read()` gives simple data structures.

Method Call

Method Call

```
data = self.env['student.student'].browse(15).read([
    'name',
    'birth_date',
    'gender',
])
```

- Here, `.read(...)` is the **method call**.
- Return type reminder: **list of dicts**, not a recordset.

`search_read()` ORM Method

`search_read()` Method

- The **`search_read()`** method combines `search()` and `read()` in one call.
- It finds records with a domain, then returns their field values as dictionaries.
- It is heavily used by the Odoo web client.

Method Syntax

```
@api.model
def search_read(self, domain=None, fields=None,
        offset=0, limit=None, order=None):
  return super().search_read(domain=domain, fields=fields,
    offset=offset, limit=limit, order=order)
```

Breaking Down the Method Signature

- **domain**: Filter conditions (same format as `search()`).
- **fields**: List of field names to return.
- **offset**: Number of records to skip.
- **limit**: Maximum number of records.
- **order**: Sort expression.

```
domain = [('active', '=', True)]
fields = ['name', 'age', 'email']
```

What Does search_read() Return?

- It returns a **list of dictionaries**.

```
data = self.env['student.student'].search_read(
    [('age', '>=', 18)],
    ['name', 'age'],
    limit=2,
    order='age desc',
)
print(data)
```

Output is (example):

```
[
    {'id': 22, 'name': 'Amina', 'age': 25},
    {'id': 15, 'name': 'Ahmed', 'age': 22},
]
```

SEARCH_READ() VS SEARCH() + READ()

Equivalent ideas:

```python
# Two calls
records = self.env['student.student'].search(domain, limit=10)
data = records.read(['name', 'age'])

# One call
data = self.env['student.student'].search_read(
  domain, ['name', 'age'], limit=10,
)
```

- `search_read()` is convenient and often more efficient for RPC.
- Inside server business logic, `search()` + recordset access is usually clearer.

When to Use `search_read()`

- Building API responses.
- Returning table/list data to the UI.
- Exporting selected fields without keeping a recordset around.

```
rows = self.env['student.student'].search_read(
    [('gender', '=', 'female')],
    ['name', 'email', 'birth_date'],
    order='name asc',
)
```

Method Call

Method Call

```
rows = self.env['student.student'].search_read(
    [('active', '=', True)],
    fields=['name', 'age'],
    offset=0,
    limit=50,
    order='id desc',
)
```

- Here, `.search_read(...)` is the **method call**.
- Remember: return type is a list of dicts.

`search_count()` ORM Method

`search_count()` Method

- The **`search_count()`** method returns the **number of records** that match a domain.
- It is more efficient than searching all records and then using `len()`.
- The method returns an integer.

Method Syntax

```
@api.model
def search_count(self, domain, limit=None):
    return super().search_count(domain, limit=limit)
```

- Important parts: **`domain`**, optional **`limit`**.

Breaking Down the Method Signature

- **`@api.model`**: Executed at the model level.
- **`self`**: The model, not a specific record.
- **`domain`**: Same domain language used by `search()`.
- **`limit`**: Optional maximum count (less commonly used).

```
domain = [('active', '=', True), ('age', '>=', 18)]
```

What Does search_count() Return?

- It returns an **integer**.

```
total = self.env['student.student'].search_count([
  ('gender', '=', 'male'),
])
print(total)
```

Output is (example):

```
12
```

- No recordset is created.
- Only the count is computed.

Why Not Use `len(search(...))`?

Less efficient pattern:

```
total = len(self.env['student.student'].search([
  ('active', '=', True),
]))
```

Better pattern:

```
total = self.env['student.student'].search_count([
  ('active', '=', True),
])
```

- `search_count()` avoids loading every matching record into Python.
- Important when tables are large.

Practical Examples

```python
adults = self.env['student.student'].search_count([
    ('age', '>=', 18),
])
archived = self.env['student.student'].search_count([
    ('active', '=', False),
])
has_email = self.env['student.student'].search_count([
    ('email', '!=', False),
])
```

- Useful for dashboards, KPIs, and validation checks.

Method Call

Method Call

```
count = self.env['student.student'].search_count([
    ('name', 'ilike', 'ahmed'),
])
```

- Here, `.search_count(...)` is the **method call**.
- Return type reminder: **int**.

`name_create()` ORM Method

NAME_CREATE() METHOD

- The **name_create()** method creates a record using only a display name.
- It is used by many2one fields when the user types a new value and quick-creates it.
- Internally, it usually calls `create()` with the `name` field.

Method Syntax

```
@api.model
def name_create(self, name):
    return super().name_create(name)
```

- Important parts: **@api.model**, **name**.

Breaking Down the Method Signature

- **`@api.model`**: Runs at the model level.
- **`self`**: The model on which the new record is created.
- **`name`**: A string that becomes the record's name / display name.

```
name = "Ahmed Mohamed"
```

- No full `vals` dictionary is required by the caller.
- Extra required fields must be handled by defaults or an override.

What Does `name_create()` Return?

- It returns a tuple: `(id, display_name)`.

```
result = self.env['student.student'].name_create('Ahmed Mohamed')
print(result)
```

Output is (example):

```
(29, 'Ahmed Mohamed')
```

- First value: database ID of the new record.
- Second value: display name shown in the UI.

How the UI Uses `name_create()`

- In a many2one field, the user types a name that does not exist.
- Odoo offers “Create . . . ” / quick create.
- Selecting that option calls `name_create()`.

```
# Conceptual flow
# typed value: "Grade 12"
self.env['school.class'].name_create('Grade 12')
```

- This is why many2one quick-create only needs a name.

Override Pattern

When a model has more required fields than `name`:

```
@api.model
def name_create(self, name):
    record = self.create({
        'name': name,
        'active': True,
        'gender': 'other',
    })
    return record.id, record.display_name
```

- Override `name_create()` when quick-create must fill extra values.

Method Call

Method Call

```
record_id, display_name = self.env['student.student'].name_create(
    'Ahmed Mohamed'
)
```

- Here, `.name_create(...)` is the **method call**.
- Return type reminder: **tuple (id, display_name)**.

`default_get()` ORM Method

`default_get()` Method

- The **`default_get()`** method returns the **default values** used when creating a new record in the UI or through the ORM.
- It collects defaults from field definitions, context, and company settings.
- The method returns a dictionary of field values.

Method Syntax

```
@api.model
def default_get(self, fields_list):
    return super().default_get(fields_list)
```

- Important part: **`fields_list`**.

Breaking Down the Method Signature

- **`@api.model`**: Model-level method.
- **`self`**: The model requesting defaults.
- **`fields_list`**: List of field names for which defaults are needed.

```
fields_list = ['name', 'gender', 'active', 'company_id']
```

- The UI calls this when you click **New** on a form.

What Does `default_get()` Return?

- It returns a dictionary of default values.

```
defaults = self.env['student.student'].default_get([
    'name', 'gender', 'active',
])
print(defaults)
```

Output is (example):

```
{'active': True, 'gender': 'male'}
```

- Only fields that have defaults appear in the dictionary.
- Missing keys mean “no default”.

Where Defaults Come From

1. Field-level default:

```
active = fields.Boolean(default=True)
gender = fields.Selection(..., default='male')
```

2. Context default:

```
self.env['student.student'].with_context(
    default_gender='female'
).default_get(['gender'])
```

3. Custom logic inside an overridden `default_get()`.

Override Pattern

```python
@api.model
def default_get(self, fields_list):
    defaults = super().default_get(fields_list)
    if 'name' in fields_list:
        defaults['name'] = 'New Student'
    if 'active' in fields_list:
        defaults['active'] = True
    return defaults
```

- Always call `super()` first, then adjust the dictionary.
- Only set keys that were requested in `fields_list`.

Method Call

Method Call

```
vals = self.env['student.student'].default_get([
    'name', 'birth_date', 'gender', 'active',
])
```

- Here, `.default_get(...)` is the **method call**.
- Return type reminder: **dict**.

`name_search()` ORM Method

`name_search()` Method

- The **`name_search()`** method searches records by name / display name.
- It powers many2one dropdown search-as-you-type behavior.
- It returns a list of `(id, display_name)` tuples.
- To customize search logic, developers often override **`_name_search()`**.

Method Syntax

```
@api.model
def name_search(self, name='', args=None,
                operator='ilike', limit=100):
    return super().name_search(name=name, args=args,
        operator=operator, limit=limit)
```

Breaking Down the Method Signature

- **`name`**: Text typed by the user.
- **`args`**: Extra domain restrictions.
- **`operator`**: Comparison operator, usually `ilike`.
- **`limit`**: Maximum number of suggestions.

```
name = 'ahm'
args = [('active', '=', True)]
operator = 'ilike'
limit = 8
```

What Does `name_search()` Return?

- It returns a list of tuples: `(id, display_name)`.

```
result = self.env['student.student'].name_search('ahm')
print(result)
```

Output is (example):

```
[
  (15, 'Ahmed Mohamed'),
  (22, 'Ahmed Ali'),
]
```

- Perfect format for many2one selection lists.

How Many2one Uses name_search()

- User opens a many2one field and types text.
- Odoo calls `name_search()` with that text.
- Matching records appear in the dropdown.

```
self.env['student.student'].name_search(
    name='amina',
    args=[('active', '=', True)],
    operator='ilike',
    limit=10,
)
```

Searching Extra Fields

By default, name search focuses on the name / display name. You can extend it:

```
@api.model
def _name_search(self, name='', args=None,
        operator='ilike', limit=100, order=None):
  args = args or []
  domain = ['|',
    ('name', operator, name),
    ('email', operator, name)]
  return self._search(domain + args, limit=limit, order=order)
```

- Now users can find a student by name or email.
- `_name_search()` is the private method commonly overridden.

Method Call

Method Call

```
matches = self.env['student.student'].name_search(
    name='ahmed',
    args=[('gender', '=', 'male')],
    operator='ilike',
    limit=5,
)
```

- Here, `.name_search(...)` is the **method call**.
- Return type reminder: **list of (id, display_name)**.

`get_view()` ORM Method

`get_view()` Method

- The **`get_view()`** method returns the architecture and metadata of a view for a model (form, list/tree, kanban, search, ...).
- The web client uses it to know how to render a screen.
- In Odoo 17, `get_view()` replaces older multi-view helpers such as `fields_view_get()` in many flows.

Method Syntax

```
@api.model
def get_view(self, view_id=None, view_type='form', **options):
    return super().get_view(view_id=view_id, view_type=view_type, **options)
```

Breaking Down the Method Signature

- **view_id**: Optional specific view XML ID / database ID.
- **view_type**: Type of view to load: `'form'`, `'list'`, `'kanban'`, `'search'`, ...
- **options**: Extra options used by the client / ORM.

```
view_type = 'form'
view_id = None  # let Odoo choose the default form view
```

What Does `get_view()` Return?

- It returns a dictionary of view information.

```
info = self.env['student.student'].get_view(view_type='form')
print(info.keys())
```

Common keys include:

- `id` – view record ID
- `name` – view name
- `type` – view type
- `arch` – the XML architecture
- `models` – field/model metadata needed by the view

Practical Example

```
form_view = self.env['student.student'].get_view(view_type='form')
list_view = self.env['student.student'].get_view(view_type='list')

print(form_view['type'])    # form
print(list_view['type'])    # list
print(form_view['arch'])    # XML architecture
```

- Developers rarely call this in business methods.
- It is essential for UI, studio-like tools, and technical debugging.

When Is `get_view()` Useful?

- Understanding which view Odoo will open for a model.
- Building custom clients that render Odoo views.
- Debugging inherited / extended view architecture.
- Inspecting the final combined XML after view inheritance.

```
info = self.env['student.student'].get_view(view_type='search')
```

Method Call

Method Call

```
view_info = self.env['student.student'].get_view(
    view_id=None,
    view_type='form',
)
```

- Here, `.get_view(...)` is the **method call**.
- Return type reminder: **dict** of view metadata.

`ensure_one()` ORM Method

`ensure_one()` Method

- The **`ensure_one()`** method verifies that a recordset contains **exactly one** record.
- If the recordset is empty or contains multiple records, it raises an error.
- It returns the same recordset when the check passes.

Method Syntax

```
def ensure_one(self):
    return super().ensure_one()
```

- Important parts: **`ensure_one`**, **`self`**.

Breaking Down the Method Signature

- **ensure_one**: Safety check method for singleton recordsets.
- **self**: The recordset being validated.
- No extra arguments are required.

```
student = self.env['student.student'].browse(15)
student.ensure_one()
```

- This succeeds because there is exactly one record.

What Does ensure_one() Return?

- On success, it returns the same recordset.

```
student = self.env['student.student'].browse(15)
result = student.ensure_one()
print(result)
```

Output is:

```
student.student(15,)
```

- On failure, it raises `ValueError`.

When It Fails

Empty recordset:

```
students = self.env['student.student'].browse([])
students.ensure_one()  # ValueError
```

Multiple records:

```
students = self.env['student.student'].browse([15, 16])
students.ensure_one()  # ValueError
```

- Error message idea: “Expected singleton”.

Why Is `ensure_one()` Important?

- Many methods are written for one record only (send email, print report, open wizard).
- Accessing `record.name` on a multi-recordset can raise singleton errors later.
- Calling `ensure_one()` makes the expectation explicit and fails early.

```python
def action_confirm(self):
    self.ensure_one()
    self.state = 'confirmed'
    return True
```

Method Call

Method Call

```
1 self.ensure_one()
```

- Here, **`.ensure_one()`** is the **method call**.
- Use it at the start of button actions and record-specific business methods.

`filtered()` ORM Method

`filtered()` Method

- The **`filtered()`** method filters an existing recordset in Python.
- Unlike `search()`, it does **not** run a new SQL query by domain.
- It returns a new recordset containing only records that match a condition.

Method Syntax

```
def filtered(self, func):
    return super().filtered(func)
```

- Important parts: **`self`**, **`func`**.

Breaking Down the Method Signature

- **`self`**: The starting recordset.
- **`func`**: Either
  - a function / lambda that returns True/False, or
  - a field name string (truthy check).

```
students.filtered(lambda s: s.age >= 18)
students.filtered('email')    # keep records with a truthy email
```

What Does `filtered()` Return?

- It returns a recordset.

```
students = self.env['student.student'].search([])
adults = students.filtered(lambda s: s.age >= 18)
print(adults)
```

Output is (example):

```
student.student(15, 22)
```

Filtered with a Lambda

```
students = self.env['student.student'].browse([15, 16, 17])

males = students.filtered(lambda s: s.gender == 'male')
with_email = students.filtered(lambda s: bool(s.email))
active_adults = students.filtered(
  lambda s: s.active and s.age >= 18
)
```

- The lambda receives one record at a time.
- Return `True` to keep the record.

FILTERED() VS SEARCH()

- Use `search()` when filtering from the whole table with a domain (SQL).
- Use `filtered()` when you already have a recordset and need a local filter.

```python
# From database
adults = self.env['student.student'].search([('age', '>=', 18)])

# From an existing recordset
adults = students.filtered(lambda s: s.age >= 18)
```

Method Call

Method Call

```
1 adults = students.filtered(lambda student: student.age >= 18)
```

- Here, `.filtered(...)` is the **method call**.
- Return type reminder: **recordset**.

`mapped()` ORM Method

MAPPED() METHOD

- The **mapped()** method extracts values or related records from a recordset.
- It is used to map each record to a field, expression, or function result.
- The return type depends on what you map to.

Method Syntax

```
def mapped(self, func):
    return super().mapped(func)
```

- Important parts: **self**, **func**.

Breaking Down the Method Signature

- **`self`**: The source recordset.
- **`func`**: A field path string, or a function / lambda.

```
students.mapped('name')
students.mapped('classroom_id')
students.mapped('classroom_id.name')
students.mapped(lambda s: s.age * 2)
```

What Does `mapped()` Return?

Mapping a simple field returns a Python list:

```
names = students.mapped('name')
print(names)
```

```
['Ahmed', 'Amina', 'Omar']
```

Mapping a relational field returns a recordset:

```
classes = students.mapped('classroom_id')
print(classes)
```

```
school.classroom(3, 5)
```

Dot-Path Mapping

You can follow relational paths:

```
class_names = students.mapped('classroom_id.name')
teacher_names = students.mapped('classroom_id.teacher_id.name')
```

- Odoo walks the relationship for each record.
- Duplicate relational records are compressed in the resulting recordset.
- This is cleaner than writing manual loops.

Mapping with a Function

```
labels = students.mapped(
    lambda s: f"{s.name} ({s.age})"
)
ages = students.mapped(lambda s: s.age)
total_age = sum(students.mapped('age'))
```

- Function mapping always returns a list.
- Very useful for reports, totals, and display labels.

Method Call

Method Call

```
emails = students.mapped('email')
classrooms = students.mapped('classroom_id')
```

- Here, `.mapped(...)` is the **method call**.
- Return type: **list** or **recordset**, depending on the mapped path.

`sorted()` ORM Method

sorted() Method

- The **sorted()** method returns a recordset ordered in Python.
- It sorts an existing recordset; it does not query the database again.
- The method returns a new sorted recordset.

Method Syntax

```
def sorted(self, key=None, reverse=False):
    return super().sorted(key=key, reverse=reverse)
```

- Important parts: **key**, **reverse**.

Breaking Down the Method Signature

- **`self`**: The recordset to sort.
- **`key`**: A field name string, or a function that returns the sort value.
- **`reverse`**: `False` for ascending, `True` for descending.

```
students.sorted('name')
students.sorted(key=lambda s: s.age, reverse=True)
```

What Does sorted() Return?

- It returns a recordset in the new order.

```
students = self.env['student.student'].browse([17, 15, 16])
ordered = students.sorted('name')
print(ordered)
```

Output is (example):

```
student.student(15, 16, 17)
```

Sorting by Field or Lambda

```
by_name = students.sorted('name')
by_age_desc = students.sorted('age', reverse=True)
by_email = students.sorted(key=lambda s: s.email or '')
by_class_then_name = students.sorted(
  key=lambda s: (s.classroom_id.name or '', s.name or '')
)
```

- Use a lambda when you need multi-key or computed sorting.

SORTED() VS SEARCH(... ORDER=)

- `search(..., order='age desc')` sorts in SQL while fetching.
- `sorted()` sorts an already loaded recordset in Python.

```python
# SQL ordering
students = self.env['student.student'].search([], order='age desc')

# Python ordering
students = students.sorted('age', reverse=True)
```

Method Call

Method Call

```
1 ordered_students = students.sorted(key='age', reverse=True)
```

- Here, `.sorted(...)` is the **method call**.
- Return type reminder: **recordset**.

`grouped()` ORM Method

`grouped()` Method

- The **`grouped()`** method groups an existing recordset by a field or key.
- It is available on recordsets in modern Odoo versions (including Odoo 17).
- It returns a dictionary mapping each group key to a recordset.

Method Syntax

```
def grouped(self, key):
    return super().grouped(key)
```

- Important parts: **`self`**, **`key`**.

Breaking Down the Method Signature

- **`self`**: The recordset to group.
- **`key`**: A field name string, or a function that returns the group key.

```
students.grouped('gender')
students.grouped('classroom_id')
students.grouped(lambda s: s.age // 10)  # age decade groups
```

What Does grouped() Return?

- It returns a dictionary.

```
students = self.env['student.student'].search([])
groups = students.grouped('gender')
print(groups)
```

Output is (example):

```
{
    'male': student.student(15, 17),
    'female': student.student(16, 22),
}
```

Grouping by a Many2one Field

```
grouped_by_class = students.grouped('classroom_id')

for classroom, class_students in grouped_by_class.items():
    print(classroom.name, len(class_students))
```

- Keys are relational records (or empty record / falsy key when empty).
- Values are recordsets of students in that group.

Practical Use Cases

- Build totals per group without writing SQL `GROUP BY`.
- Split records before batch processing.
- Prepare data for reports and dashboards.

```
groups = students.grouped('gender')
male_count = len(groups.get('male', self.env['student.student']))
female_count = len(groups.get('female', self.env['student.student']))
```

Method Call

Method Call

```
groups = students.grouped('classroom_id')
```

- Here, `.grouped(...)` is the **method call**.
- Return type reminder: **dict[key → recordset]**.

`fields_get()` ORM Method

`fields_get()` Method

- The **`fields_get()`** method returns **metadata** about model fields.
- It describes field type, string/label, required, selection values, relation, etc.
- The web client uses it to understand how fields should behave in the UI.

Method Syntax

```
@api.model
def fields_get(self, allfields=None, attributes=None):
    return super().fields_get(allfields=allfields, attributes=attributes)
```

Breaking Down the Method Signature

- **allfields**: Optional list of field names to inspect. If omitted, metadata for all fields is returned.
- **attributes**: Optional list of attribute names to include (for example `string`, `type`, `required`).

```
self.env['student.student'].fields_get(['name', 'gender'])
self.env['student.student'].fields_get(
    ['age'], attributes=['string', 'type', 'required']
)
```

What Does `fields_get()` Return?

- It returns a dictionary of dictionaries.

```
info = self.env['student.student'].fields_get(['name', 'gender'])
print(info)
```

Output is (example):

```
{
    'name': {'type': 'char', 'string': 'Name', 'required': True},
    'gender': {
        'type': 'selection',
        'string': 'Gender',
        'selection': [('male', 'Male'), ('female', 'Female')],
    },
}
```

Useful Metadata Keys

Common keys you will see:

- `type` – field type (`char`, `many2one`, ...)
- `string` – field label
- `required` – whether the field is required
- `readonly` – whether the field is read-only
- `help` – help text
- `selection` – values for selection fields
- `relation` – comodel name for relational fields

Practical Example

```python
meta = self.env['student.student'].fields_get(
    ['name', 'classroom_id', 'gender'],
    attributes=['string', 'type', 'required', 'relation', 'selection'],
)

print(meta['classroom_id']['type'])        # many2one
print(meta['classroom_id']['relation'])    # school.classroom
print(meta['gender']['selection'])
```

- Very useful for dynamic UI, generic exports, and technical debugging.

Method Call

Method Call

```
fields_info = self.env['student.student'].fields_get(
    allfields=['name', 'age', 'email'],
    attributes=['string', 'type', 'required'],
)
```

- Here, `.fields_get(...)` is the **method call**.
- Return type reminder: **dict**.

`get_metadata()` ORM Method

GET_METADATA() METHOD

- The **get_metadata()** method returns technical metadata about records.
- It includes information such as XML IDs, create/write audit fields, and whether the record is a noupdate data record.
- It is mostly used by technical tools and the UI debug / metadata dialogs.

Method Syntax

```
def get_metadata(self):
    return super().get_metadata()
```

- Important parts: **get_metadata**, **self**.

Breaking Down the Method Signature

- **`self`**: The recordset to inspect.
- No values dictionary is required.
- Works on one or many records; returns one metadata dict per record.

```
student = self.env['student.student'].browse(15)
meta = student.get_metadata()
```

What Does `get_metadata()` Return?

- It returns a **list of dictionaries**.

```
meta = self.env['student.student'].browse(15).get_metadata()
print(meta)
```

Output is (example):

```
[{
    'id': 15,
    'xmlid': False,
    'noupdate': False,
    'create_uid': (1, 'Administrator'),
    'create_date': '2026-07-01 08:12:33',
    'write_uid': (2, 'Ahmed Muhumed'),
    'write_date': '2026-07-20 14:05:10',
}]
```

Important Metadata Keys

- `id` – database ID
- `xmlid` – external ID if the record came from XML/data files
- `noupdate` – whether upgrades should skip updating this data record
- `create_uid` / `create_date` – who created it and when
- `write_uid` / `write_date` – who last modified it and when

When to Use get_metadata()

- Debugging module data and external IDs.
- Checking whether a record is system/data XML versus user-created.
- Inspecting create/write audit information quickly.

```
for row in self.env['student.student'].browse([15, 16]).get_metadata():
    print(row['id'], row['xmlid'], row['write_date'])
```

Method Call

Method Call

```
1 metadata = self.env['student.student'].browse(15).get_metadata()
```

- Here, `.get_metadata()` is the **method call**.
- Return type reminder: **list of dicts**.

Summary

- The Odoo ORM lets you work with PostgreSQL through Python recordsets.
- **`create` / `write` / `unlink` / `copy`** manage the life cycle of records.
- **`search` / `read` / `search_read` / `search_count`** query and fetch data.
- **`name_create` / `name_search` / `default_get` / `get_view` / `fields_get` / `get_metadata`** support UI and metadata workflows.
- **`exists` / `ensure_one` / `filtered` / `mapped` / `sorted` / `grouped`** help you work safely with recordsets.